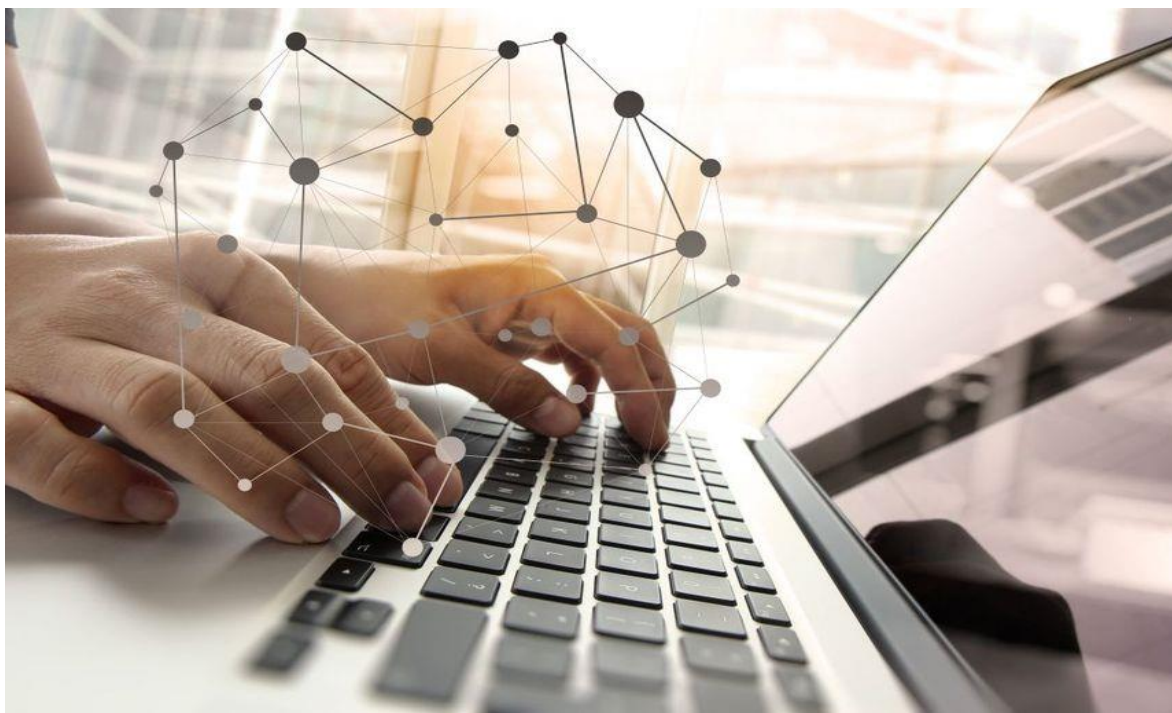




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Análisis y Diseño de Software I	Unidad 4	Tema: Funcionalidades del sistema a desarrollar.
	Semana 11	Instructora: Katya Michelle Asencio Bernal

Introducción:

Bienvenido al tercer material de lectura de la Unidad #4 de Análisis y Diseño de Software I. En este material se estudiará el proceso de propuesta de funcionalidades del sistema. Se abordará la identificación y documentación de las funciones que realizará el sistema, así como la elaboración de documentos entregables relacionados con dichas funcionalidades. Además, se desarrollarán temas como el diccionario de datos y los manuales de usuario, analista y programador.

Este material de lectura busca brindarte una base sólida en campo del Análisis y Diseño de Software, preparándote para enfrentar los desafíos y oportunidades que encontrarás en tu carrera profesional. A medida que avances en este material, te alentamos a participar activamente de las actividades, para favorecer tu proceso de aprendizaje.

Antes de comenzar, asegúrate de tener papel y lápiz a mano para las actividades interactivas.

Sección 1:

4.3 Proponer las funcionalidades del sistema a desarrollar.

Las funcionalidades del sistema representan las acciones, procesos y servicios que el sistema será capaz de realizar para satisfacer las necesidades de los usuarios y de la organización. Estas funcionalidades describen el comportamiento esperado del software y permiten establecer claramente qué tareas podrá ejecutar el sistema.

Definir correctamente las funcionalidades permitirá:

- Comprender el alcance del sistema.
- Identificar necesidades del usuario.
- Evitar ambigüedades durante el desarrollo.
- Facilitar la comunicación entre analistas, clientes y programadores.
- Servir como base para el diseño y la programación.

Las funcionalidades se clasifican entre funcionalidades principales y secundarias:

Las **funcionalidades principales** son aquellas funciones esenciales y críticas que permiten que el sistema cumpla con su objetivo principal. Estas funcionalidades están directamente relacionadas con las necesidades más importantes del usuario y con los procesos principales de la organización.

Por otro lado, las **funcionalidades secundarias** son aquellas funciones complementarias que apoyan y mejoran el funcionamiento general del sistema, pero que no constituyen el objetivo principal del software. Aunque son importantes, el sistema puede seguir operando sin ellas.

Proceso para proponer funcionalidades del sistema

Proponer las funcionalidades de un sistema consiste en identificar, analizar y definir las acciones y procesos que el



Actividad Interactiva #1

Verdadero o falso

- Las funcionalidades principales son esenciales para el sistema. ()
- Las funcionalidades secundarias ayudan a complementar el sistema. ()
- Las funcionalidades del sistema se identifican únicamente durante la programación. ()
- Las entrevistas y observaciones ayudan a proponer funcionalidades del sistema. ()

software deberá realizar para satisfacer las necesidades de los usuarios y de la organización.

Etapas para proponer funcionalidades

1. **Recolección de información:** Consiste en obtener información sobre cómo funciona actualmente la organización o proceso.
2. **Análisis de procesos:** Se estudian actividades que realiza la organización para comprender cómo fluye la información.
3. **Identificación de necesidades:** En esta etapa se determinan las funciones que el usuario necesita para mejorar sus actividades.
4. **Definición de funcionalidades:** Se redactan las funciones que tendrá el sistema. Las funciones deben describirse de forma: clara, específica, comprensible y organizada.
5. **Priorización de funcionalidades:** Las funcionalidades se organizan según su importancia. Generalmente se clasifican en: Funcionalidades principales, funcionalidades secundarias y funciones opcionales.
6. **Validación con los usuarios:** Las funciones propuestas deben revisarse con el cliente o usuarios finales para verificar que realmente solucionen las necesidades identificadas.

Técnicas utilizadas para proponer funcionalidades

Existen diferentes técnicas que ayudan al analista a recopilar información y definir correctamente las funcionalidades del sistema. Entre ellas están:

- Entrevistas
- Encuestas o cuestionarios
- Observación
- Revisión documental
- Lluvia de ideas
- Casos de uso

Recomendaciones para proponer funcionalidades

- Escuchar atentamente al usuario.
- Comprender el problema antes de proponer soluciones.
- Evitar funcionalidades innecesarias.
- Priorizar procesos importantes.
- Documentar correctamente cada funcionalidad.
- Validar constantemente con el cliente.

Sección 2:

4.3.1 Identificar y elaborar los documentos entregables.

Una vez que se identifican y proponen las funcionalidades del sistema, es necesario documentarlas correctamente para que todos los involucrados comprendan cómo funcionará el software.

Los documentos entregables permiten registrar, organizar y comunicar las funcionalidades del sistema de manera clara y estructurada.

Gracias a estos documentos:

- Los analistas comprenden los procesos.
- Los programadores saben qué desarrollar.
- Los usuarios validan las funcionalidades.
- Los testers verifican el funcionamiento.
- La empresa mantiene evidencia del proyecto.

Elaborar los documentos entregables implica crear documentación clara, organizada y útil para los diferentes participantes del proyecto. Un documento entregable debe de:

- Redactarse con lenguaje comprensible.
- La información debe tener una estructura lógica
- Debe evitar ambigüedades
- Incluir toda la información necesaria
- Mantenerse actualizados todos los cambios en el sistema
- Ser fácil de consulta y de localizar información rápidamente

Principales documentos entregables en la etapa de proponer funciones del sistema

Documento de requerimientos (SRS)

Describe las necesidades y funciones que tendrá el sistema:

Incluye:

- Requerimientos funcionales
- Requerimientos no funcionales
- Restricciones
- Objetivos
- Etc.

Diagramas del sistema

Representan gráficamente procesos y componentes.

Ejemplos:

- Diagramas de flujo.
- Diagramas UML.
- Casos de uso.
- Diagramas entidad-relación.

Prototipos de interfaz

Representan visualmente cómo será el sistema.

Permiten:

- Validar diseño.
- Mejorar experiencia del usuario,
- Detectar errores tempranos.

Manuales

Documentos de apoyo para usuarios y personal técnico.

Ejemplos:

- Manual de usuario.
- Manual técnico.
- Manual del programador.

Sección 3:

4.3.2 Diccionario de datos

Cada funcionalidad del sistema necesita trabajar con información específica. Para que el software funcione correctamente, es necesario definir claramente qué datos utilizará cada proceso, cómo se almacenarán y qué significado tendrá cada elemento de información.

El diccionario de datos es un documento que describe detalladamente todos los datos utilizados dentro de un sistema.

Funciona como un catálogo de información donde se especifican las características de cada dato que utilizarán las funcionalidades del sistema.

EL diccionario de datos ayuda a organizar y estandarizar la información para que analistas, programadores y administradores comprendan exactamente cómo se manejarán los datos.

Elementos de un diccionario de datos

Nombre del campo

Identifica el dato dentro del sistema.

Ejemplos:

- id_usuario
- nombre_cliente
- fecha_pago

Descripción

Explica el propósito o significado del dato.

Ejemplo:

- "Identificador único del estudiante".

Tipo de dato

Define qué tipo de información almacenará.

Ejemplos:

- Entero



Actividad Interactiva #2

Caso práctico

La funcionalidad "Registrar estudiantes" necesita almacenar información.

Escribe 3 datos necesarios para esta funcionalidad.

This image shows a blank sheet of handwriting practice paper. It features ten sets of horizontal dashed lines, each set consisting of three parallel lines. These lines are designed to help children learn letter height and placement while practicing their writing skills. The background is white, and the lines are light gray.

- Texto
- Decimal
- Fecha
- Booleano

Longitud

Indica el tamaño permitido para almacenar el dato.

Ejemplos:

- VARCHAR(50)
- INT(10)

Restricciones

Definen reglas o condiciones que debe cumplir el dato.

Ejemplos:

- No nulo
- Único
- Llave primaria
- Llave foránea

Ejemplo de diccionario de datos:

Campo	Tipo	Tamaño	Descripción	Restricción
id_estudiante	INT	10	Identificador del estudiante	Llave primaria
nombre	VARCHAR	50	Nombre completo	No nulo
correo	VARCHAR	100	Correo electrónico	Único
fecha_nacimiento	DATE	-	Fecha de nacimiento	No nulo

[illegible]

Manual del analista

Es un documento técnico que describe el análisis y diseño funcional del sistema. Está dirigido a los analistas, diseñadores y personal encargado de mantenimiento y mejoras.

Los principales objetivos del manual del analista son:

- Documentar funcionalidades del sistema.
- Explicar procesos y reglas de negocio.
- Facilitar futuras modificaciones.
- Registrar estructura lógica del software.

Contenido del manual del analista:

- **Descripción del sistema:** Objetivos y alcance
- **Requerimientos funcionales:** Funciones que realizará el sistema.
- **Requerimientos no funcionales:** Condiciones técnicas y de calidad.
- **Diagramas:** Casos de uso, diagramas UML.
- **Reglas del negocio:** Normas que debe cumplir el sistema.
- **Estructura de datos:** Información relacionada con la base de datos.

Manual del programador

Es un documento técnico dirigido a desarrolladores encargados del código fuente y mantenimiento del sistema.

Describe cómo fueron implementadas las funcionalidades del software.

Entre los objetivos del manual del programador están:

- Explicar la estructura técnica del sistema.
- Facilitar mantenimiento del código.
- Ayudar a nuevos desarrolladores.
- Documentar la implementación de funcionalidades.

Contenido del manual del programador:

- **Lenguajes utilizados:** Ejemplos: Java, Python, PHP.
- **Arquitectura del sistema**
- **Organización del proyecto:** Carpetas, módulos y archivos.
- **Base de datos:** Configuración y conexión.
- **Librerías y dependencias.**
- **Explicación del código:** Funciones y módulos principales.
- **Instalación y configuración:** Pasos para ejecutar el sistema.

Bibliografía

Sommerville, I. (2011). Ingeniería de software (9th ed.). Pearson Educación de México,

S. A. de C. V.

Guía de manual de usuario: Cómo crear en línea, herramientas y mejores prácticas.

(2026, February 25). Document360.

<https://document360.com/es/blog/creando-un-manual-de-usuario/>

Kendall, K. E., & Kendall, J. E. (2011). *Análisis y diseño de sistemas*. Pearson

Educación.

